01. Inicio de sesión

Tipo CQRS: Comando (Command): operación que modifica el estado del sistema.

Class Name: InicioSesionComponent	
Responsibilities Capturar usuario y contraseña; mostrar errores y sesión iniciada.	**Collaborators** InicioSesionViewModel

Class Name: InicioSesionViewModel	
Responsibilities Mantener usuario, contraseña, loading y mensajes.	**Collaborators** InicioSesionFacade

Class Name: InicioSesionFacade	
Responsibilities Validar datos visuales, llamar al cliente y guardar la sesión.	**Collaborators** AuthApiClient, SessionStore, Router

Class Name: AuthApiClient	
Responsibilities Enviar POST /api/auth/login.	**Collaborators** AuthController

Class Name: AuthController	
Responsibilities Recibir credenciales y crear el comando.	**Collaborators** IniciarSesionCommand, IniciarSesionHandler

Class Name: IniciarSesionCommand	
Responsibilities Transportar usuario y contraseña.	**Collaborators** Ninguno

Class Name: IniciarSesionValidator	
Responsibilities Validar campos requeridos y longitud de contraseña.	**Collaborators** IValidator

Class Name: IniciarSesionHandler	
Responsibilities Buscar empleado, verificar contraseña y generar token.	**Collaborators** IEmpleadoRepository, IPasswordHasher, ITokenService

Class Name: EmpleadoRepository	
Responsibilities Consultar el empleado en la base de datos.	**Collaborators** TacomandaDbContext

Recorrido de los datos

InicioSesionComponent → InicioSesionViewModel → InicioSesionFacade → AuthApiClient → AuthController → IniciarSesionCommand → IniciarSesionValidator → IniciarSesionHandler → EmpleadoRepository → Base de datos.

Regreso: el resultado se devuelve por el Handler, Controller, API Client y Facade hasta actualizar el ViewModel y la pantalla.

02. Crear comanda

Tipo CQRS: Comando (Command): operación que modifica el estado del sistema.

Class Name: CrearComandaComponent	
Responsibilities Seleccionar mesa, platillos y cantidades.	**Collaborators** CrearComandaViewModel

Class Name: CrearComandaViewModel	
Responsibilities Mantener carrito, mesa seleccionada y total.	**Collaborators** CrearComandaFacade

Class Name: CrearComandaFacade	
Responsibilities Preparar la comanda y enviarla.	**Collaborators** ComandasApiClient

Class Name: ComandasApiClient	
Responsibilities Enviar POST /api/comandas.	**Collaborators** ComandasController

Class Name: ComandasController	
Responsibilities Recibir datos y crear el command.	**Collaborators** CrearComandaCommand, CrearComandaHandler

Class Name: CrearComandaCommand	
Responsibilities Transportar mesa, mesero y detalles.	**Collaborators** Ninguno

Class Name: CrearComandaValidator	
Responsibilities Validar IDs, cantidades y observaciones.	**Collaborators** IValidator

Class Name: CrearComandaHandler	
Responsibilities Verificar mesa, platillos y mesero; crear la comanda.	**Collaborators** IMesaRepository, IPlatilloRepository, IComandaRepository

Class Name: ComandaRepository	
Responsibilities Guardar la comanda y sus detalles.	**Collaborators** TacomandaDbContext

Class Name: Comanda	
Responsibilities Proteger reglas como cantidades válidas y total.	**Collaborators** DetalleComanda, Mesa

Recorrido de los datos

CrearComandaComponent → CrearComandaViewModel → CrearComandaFacade → ComandasApiClient → ComandasController → CrearComandaCommand → CrearComandaValidator → CrearComandaHandler → ComandaRepository → Base de datos.

Regreso: el resultado se devuelve por el Handler, Controller, API Client y Facade hasta actualizar el ViewModel y la pantalla.

03. Dar de alta empleados

Tipo CQRS: Comando (Command): operación que modifica el estado del sistema.

Class Name: AltaEmpleadoComponent	
Responsibilities Capturar datos del nuevo empleado.	**Collaborators** AltaEmpleadoViewModel

Class Name: AltaEmpleadoViewModel	
Responsibilities Representar nombre, usuario, contraseña y rol.	**Collaborators** AltaEmpleadoFacade

Class Name: AltaEmpleadoFacade	
Responsibilities Preparar y enviar el formulario.	**Collaborators** EmpleadosApiClient

Class Name: EmpleadosApiClient	
Responsibilities Enviar POST /api/empleados.	**Collaborators** EmpleadosController

Class Name: EmpleadosController	
Responsibilities Recibir el request y crear el command.	**Collaborators** CrearEmpleadoCommand

Class Name: CrearEmpleadoCommand	
Responsibilities Transportar los datos del empleado.	**Collaborators** Ninguno

Class Name: CrearEmpleadoValidator	
Responsibilities Validar nombre, rol, usuario y contraseña.	**Collaborators** IValidator

Class Name: CrearEmpleadoHandler	
Responsibilities Verificar duplicados, proteger contraseña y registrar empleado.	**Collaborators** IEmpleadoRepository, IPasswordHasher

Class Name: EmpleadoRepository	
Responsibilities Insertar el empleado.	**Collaborators** TacomandaDbContext

Class Name: Empleado	
Responsibilities Representar al mesero o cajero y proteger sus reglas.	**Collaborators** EmployeeRole

Recorrido de los datos

AltaEmpleadoComponent → AltaEmpleadoViewModel → AltaEmpleadoFacade → EmpleadosApiClient → EmpleadosController → CrearEmpleadoCommand → CrearEmpleadoValidator → CrearEmpleadoHandler → EmpleadoRepository → Base de datos.

Regreso: el resultado se devuelve por el Handler, Controller, API Client y Facade hasta actualizar el ViewModel y la pantalla.

04. Crear platillo

Tipo CQRS: Comando (Command): operación que modifica el estado del sistema.

Class Name: CrearPlatilloComponent	
Responsibilities Capturar nombre, categoría, descripción y precio.	**Collaborators** CrearPlatilloViewModel

Class Name: CrearPlatilloViewModel	
Responsibilities Representar los datos del formulario.	**Collaborators** CrearPlatilloFacade

Class Name: CrearPlatilloFacade	
Responsibilities Preparar y enviar el nuevo platillo.	**Collaborators** PlatillosApiClient

Class Name: PlatillosApiClient	
Responsibilities Enviar POST /api/platillos.	**Collaborators** PlatillosController

Class Name: PlatillosController	
Responsibilities Recibir request y crear el command.	**Collaborators** CrearPlatilloCommand

Class Name: CrearPlatilloCommand	
Responsibilities Transportar los datos del platillo.	**Collaborators** Ninguno

Class Name: CrearPlatilloValidator	
Responsibilities Validar nombre, precio, categoría y descripción.	**Collaborators** IValidator

Class Name: CrearPlatilloHandler	
Responsibilities Aplicar reglas y registrar el platillo.	**Collaborators** IPlatilloRepository

Class Name: PlatilloRepository	
Responsibilities Guardar el platillo.	**Collaborators** TacomandaDbContext

Class Name: Platillo	
Responsibilities Proteger precio, nombre y estado activo.	**Collaborators** Ninguno

Recorrido de los datos

CrearPlatilloComponent → CrearPlatilloViewModel → CrearPlatilloFacade → PlatillosApiClient → PlatillosController → CrearPlatilloCommand → CrearPlatilloValidator → CrearPlatilloHandler → PlatilloRepository → Base de datos.

Regreso: el resultado se devuelve por el Handler, Controller, API Client y Facade hasta actualizar el ViewModel y la pantalla.

05. Editar platillo

Tipo CQRS: Comando (Command): operación que modifica el estado del sistema.

Class Name: EditarPlatilloComponent	
Responsibilities Mostrar y modificar datos del platillo.	**Collaborators** EditarPlatilloViewModel

Class Name: EditarPlatilloViewModel	
Responsibilities Mantener los valores editados.	**Collaborators** EditarPlatilloFacade

Class Name: EditarPlatilloFacade	
Responsibilities Preparar la actualización.	**Collaborators** PlatillosApiClient

Class Name: PlatillosController	
Responsibilities Recibir PUT /api/platillos/{id}.	**Collaborators** EditarPlatilloCommand

Class Name: EditarPlatilloCommand	
Responsibilities Transportar ID y nuevos valores.	**Collaborators** Ninguno

Class Name: EditarPlatilloValidator	
Responsibilities Validar los nuevos datos.	**Collaborators** IValidator

Class Name: EditarPlatilloHandler	
Responsibilities Buscar platillo, modificarlo y guardar cambios.	**Collaborators** IPlatilloRepository

Class Name: PlatilloRepository	
Responsibilities Actualizar el registro.	**Collaborators** TacomandaDbContext

Class Name: Platillo	
Responsibilities Aplicar cambios respetando sus invariantes.	**Collaborators** Ninguno

Recorrido de los datos

EditarPlatilloComponent → EditarPlatilloViewModel → EditarPlatilloFacade → PlatillosController → EditarPlatilloCommand → EditarPlatilloValidator → EditarPlatilloHandler → PlatilloRepository → Base de datos.

Regreso: el resultado se devuelve por el Handler, Controller, API Client y Facade hasta actualizar el ViewModel y la pantalla.

06. Consultar platillo

Tipo CQRS: Consulta (Query): lectura sin modificar registros.

Class Name: ConsultarPlatillosComponent	
Responsibilities Mostrar catálogo, filtros y resultados.	**Collaborators** ConsultarPlatillosViewModel

Class Name: ConsultarPlatillosViewModel	
Responsibilities Representar la lista de platillos.	**Collaborators** ConsultarPlatillosFacade

Class Name: ConsultarPlatillosFacade	
Responsibilities Solicitar y filtrar visualmente los datos.	**Collaborators** PlatillosApiClient

Class Name: PlatillosApiClient	
Responsibilities Enviar GET /api/platillos.	**Collaborators** PlatillosController

Class Name: PlatillosController	
Responsibilities Crear la consulta.	**Collaborators** ConsultarPlatilloQuery

Class Name: ConsultarPlatilloQuery	
Responsibilities Transportar filtros y paginación.	**Collaborators** Ninguno

Class Name: ConsultarPlatilloQueryHandler	
Responsibilities Consultar platillos y convertirlos a DTO.	**Collaborators** IPlatilloReader

Class Name: PlatilloReader	
Responsibilities Leer platillos desde la base de datos.	**Collaborators** TacomandaDbContext

Class Name: PlatilloDto	
Responsibilities Representar la respuesta para Angular.	**Collaborators** ConsultarPlatillosViewModel

Recorrido de los datos

ConsultarPlatillosComponent → ConsultarPlatillosViewModel → ConsultarPlatillosFacade → PlatillosApiClient → PlatillosController → ConsultarPlatilloQuery → ConsultarPlatilloQueryHandler → PlatilloReader → Base de datos.

Regreso: el resultado se devuelve por el Handler, Controller, API Client y Facade hasta actualizar el ViewModel y la pantalla.

07. Eliminar platillo

Tipo CQRS: Comando (Command): operación que modifica el estado del sistema.

Class Name: EliminarPlatilloComponent	
Responsibilities Solicitar confirmación de baja.	**Collaborators** EliminarPlatilloViewModel

Class Name: EliminarPlatilloViewModel	
Responsibilities Controlar platillo seleccionado y confirmación.	**Collaborators** EliminarPlatilloFacade

Class Name: EliminarPlatilloFacade	
Responsibilities Enviar la solicitud de baja.	**Collaborators** PlatillosApiClient

Class Name: PlatillosController	
Responsibilities Recibir DELETE /api/platillos/{id}.	**Collaborators** EliminarPlatilloCommand

Class Name: EliminarPlatilloCommand	
Responsibilities Transportar el ID del platillo.	**Collaborators** Ninguno

Class Name: EliminarPlatilloValidator	
Responsibilities Validar que el ID sea válido.	**Collaborators** IValidator

Class Name: EliminarPlatilloHandler	
Responsibilities Buscar el platillo y cambiarlo a inactivo.	**Collaborators** IPlatilloRepository

Class Name: PlatilloRepository	
Responsibilities Guardar el cambio de estado.	**Collaborators** TacomandaDbContext

Class Name: Platillo	
Responsibilities Ejecutar Deactivate().	**Collaborators** Ninguno

Recorrido de los datos

EliminarPlatilloComponent → EliminarPlatilloViewModel → EliminarPlatilloFacade → PlatillosController → EliminarPlatilloCommand → EliminarPlatilloValidator → EliminarPlatilloHandler → PlatilloRepository → Base de datos.

Regreso: el resultado se devuelve por el Handler, Controller, API Client y Facade hasta actualizar el ViewModel y la pantalla.

Regla del negocio: la eliminación del platillo se maneja como baja lógica para conservar el historial.

08. Consultar comanda

Tipo CQRS: Consulta (Query): lectura sin modificar registros.

Class Name: ConsultarComandasComponent	
Responsibilities Mostrar comandas y filtros.	**Collaborators** ConsultarComandasViewModel

Class Name: ConsultarComandasViewModel	
Responsibilities Representar comandas consultadas.	**Collaborators** ConsultarComandasFacade

Class Name: ConsultarComandasFacade	
Responsibilities Solicitar comandas y actualizar la vista.	**Collaborators** ComandasApiClient

Class Name: ComandasApiClient	
Responsibilities Enviar GET /api/comandas.	**Collaborators** ComandasController

Class Name: ComandasController	
Responsibilities Crear la consulta.	**Collaborators** ConsultarComandasQuery

Class Name: ConsultarComandasQuery	
Responsibilities Transportar filtros y estado.	**Collaborators** Ninguno

Class Name: ConsultarComandasQueryHandler	
Responsibilities Obtener y transformar comandas.	**Collaborators** IComandaReader

Class Name: ComandaReader	
Responsibilities Leer comandas y detalles.	**Collaborators** TacomandaDbContext

Class Name: ComandaDto	
Responsibilities Entregar datos a Angular.	**Collaborators** ConsultarComandasViewModel

Recorrido de los datos

ConsultarComandasComponent → ConsultarComandasViewModel → ConsultarComandasFacade → ComandasApiClient → ComandasController → ConsultarComandasQuery → ConsultarComandasQueryHandler → ComandaReader → Base de datos.

Regreso: el resultado se devuelve por el Handler, Controller, API Client y Facade hasta actualizar el ViewModel y la pantalla.

09. Cambiar estado de comanda

Tipo CQRS: Comando (Command): operación que modifica el estado del sistema.

Class Name: CambiarEstadoComandaComponent	
Responsibilities Permitir seleccionar el nuevo estado.	**Collaborators** CambiarEstadoComandaViewModel

Class Name: CambiarEstadoComandaViewModel	
Responsibilities Representar comanda y estado seleccionado.	**Collaborators** CambiarEstadoComandaFacade

Class Name: CambiarEstadoComandaFacade	
Responsibilities Enviar el cambio de estado.	**Collaborators** ComandasApiClient

Class Name: ComandasController	
Responsibilities Recibir PATCH /api/comandas/{id}/estado.	**Collaborators** CambiarEstadoComandaCommand

Class Name: CambiarEstadoComandaCommand	
Responsibilities Transportar ID y nuevo estado.	**Collaborators** Ninguno

Class Name: CambiarEstadoComandaValidator	
Responsibilities Validar ID y estado permitido.	**Collaborators** IValidator

Class Name: CambiarEstadoComandaHandler	
Responsibilities Verificar transición y actualizar comanda.	**Collaborators** IComandaRepository

Class Name: ComandaRepository	
Responsibilities Guardar el nuevo estado.	**Collaborators** TacomandaDbContext

Class Name: Comanda	
Responsibilities Validar transiciones como pendiente → preparación → lista.	**Collaborators** OrderStatus

Recorrido de los datos

CambiarEstadoComandaComponent → CambiarEstadoComandaViewModel → CambiarEstadoComandaFacade → ComandasController → CambiarEstadoComandaCommand → CambiarEstadoComandaValidator → CambiarEstadoComandaHandler → ComandaRepository → Base de datos.

Regreso: el resultado se devuelve por el Handler, Controller, API Client y Facade hasta actualizar el ViewModel y la pantalla.

Regla del negocio: la entidad Comanda autoriza las transiciones entre estados.

10. Cobrar comanda desde el rol de mesero

Tipo CQRS: Comando (Command): operación que modifica el estado del sistema.

Class Name: CobrarComandaMeseroComponent	
Responsibilities Mostrar total y capturar tipo de pago.	**Collaborators** CobrarComandaMeseroViewModel

Class Name: CobrarComandaMeseroViewModel	
Responsibilities Representar comanda, total y forma de pago.	**Collaborators** CobrarComandaMeseroFacade

Class Name: CobrarComandaMeseroFacade	
Responsibilities Preparar el cobro y mostrar resultado.	**Collaborators** CobrosApiClient

Class Name: CobrosApiClient	
Responsibilities Enviar POST /api/comandas/{id}/cobro.	**Collaborators** CobrosController

Class Name: CobrosController	
Responsibilities Recibir datos y crear el command.	**Collaborators** CobrarComandaCommand

Class Name: CobrarComandaCommand	
Responsibilities Transportar comanda, empleado y pago.	**Collaborators** Ninguno

Class Name: CobrarComandaValidator	
Responsibilities Validar monto y tipo de pago.	**Collaborators** IValidator

Class Name: CobrarComandaHandler	
Responsibilities Verificar estado, calcular total y registrar pago.	**Collaborators** IComandaRepository, IPagoRepository, IPoliticaDePago

Class Name: SelectorPoliticaPago	
Responsibilities Elegir efectivo o tarjeta.	**Collaborators** PoliticaEfectivo, PoliticaTarjeta

Class Name: Comanda	
Responsibilities Validar que pueda cerrarse.	**Collaborators** Pedido, DetalleComanda

Recorrido de los datos

CobrarComandaMeseroComponent → CobrarComandaMeseroViewModel → CobrarComandaMeseroFacade → CobrosApiClient → CobrosController → CobrarComandaCommand → CobrarComandaValidator → CobrarComandaHandler → Base de datos.

Regreso: el resultado se devuelve por el Handler, Controller, API Client y Facade hasta actualizar el ViewModel y la pantalla.

Regla del negocio: el cobro se permite al mesero autorizado y registra el pago de la comanda.

11. Editar comanda

Tipo CQRS: Comando (Command): operación que modifica el estado del sistema.

Class Name: EditarComandaComponent	
Responsibilities Mostrar detalles y permitir cambios.	**Collaborators** EditarComandaViewModel

Class Name: EditarComandaViewModel	
Responsibilities Representar cantidades, notas y platillos.	**Collaborators** EditarComandaFacade

Class Name: EditarComandaFacade	
Responsibilities Preparar la actualización.	**Collaborators** ComandasApiClient

Class Name: ComandasController	
Responsibilities Recibir PUT /api/comandas/{id}.	**Collaborators** EditarComandaCommand

Class Name: EditarComandaCommand	
Responsibilities Transportar cambios de la comanda.	**Collaborators** Ninguno

Class Name: EditarComandaValidator	
Responsibilities Validar estado, cantidades y notas.	**Collaborators** IValidator

Class Name: EditarComandaHandler	
Responsibilities Verificar que pueda editarse y actualizar detalles.	**Collaborators** IComandaRepository, IPlatilloRepository

Class Name: ComandaRepository	
Responsibilities Guardar los cambios.	**Collaborators** TacomandaDbContext

Class Name: Comanda	
Responsibilities Decidir si todavía permite modificaciones.	**Collaborators** OrderStatus, DetalleComanda

Recorrido de los datos

EditarComandaComponent → EditarComandaViewModel → EditarComandaFacade → ComandasController → EditarComandaCommand → EditarComandaValidator → EditarComandaHandler → ComandaRepository → Base de datos.

Regreso: el resultado se devuelve por el Handler, Controller, API Client y Facade hasta actualizar el ViewModel y la pantalla.